Welcome

Digital and Computer Vision

Lesson 4: Session 1

Introduction to Deep Learning & CNNs for Image Processing

Summary of the **previous** session: Morphological Operations

In the previous session we have covered :

Dilation, Erosion, Opening, and Closing.
Removing noise using morphological transformations.

Summary of the current session: Introduction to Deep Learning & CNNs for Image Processing

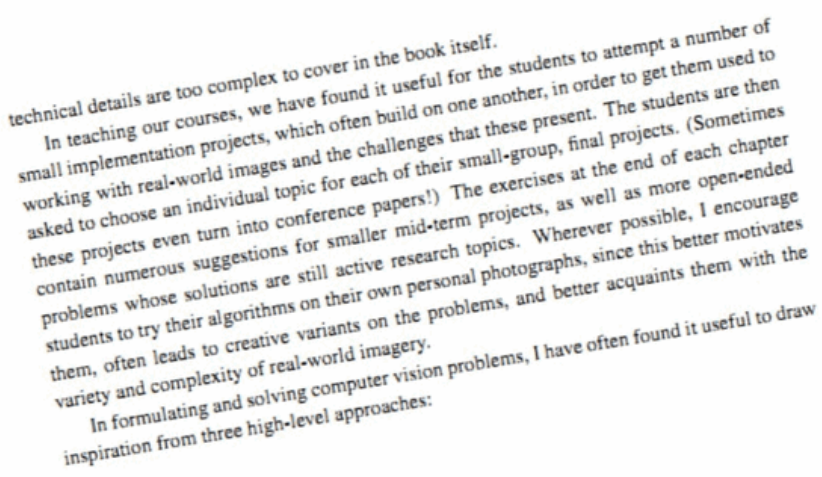
In this session we will covered the following:

Introduction to Deep Learning
Overview of Convolutional Neural Networks (CNNs).
Using TensorFlow/Keras with OpenCV.
Exercise: Building a simple CNN for image classification

Exercise from previous session 1/3

Propose a code for line detection with strongly skewed text like in the image: imageTextR.png

```
import cv2
import numpy as np

def deskew_image(gray):
    # Invert and binarize
    _, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    # Get coordinates of non-zero pixels
    coords = np.column_stack(np.where(binary > 0))

    # Get the angle of the rotated text using minAreaRect
    angle = cv2.minAreaRect(coords)[-1]
```


technical details are too complex to cover in the book itself.

In teaching our courses, we have found it useful for the students to attempt a number of small implementation projects, which often build on one another, in order to get them used to working with real-world images and the challenges that these present. The students are then asked to choose an individual topic for each of their small-group, final projects. (Sometimes these projects even turn into conference papers!) The exercises at the end of each chapter contain numerous suggestions for smaller mid-term projects, as well as more open-ended problems whose solutions are still active research topics. Wherever possible, I encourage students to try their algorithms on their own personal photographs, since this better motivates them, often leads to creative variants on the problems, and better acquaints them with the variety and complexity of real-world imagery.

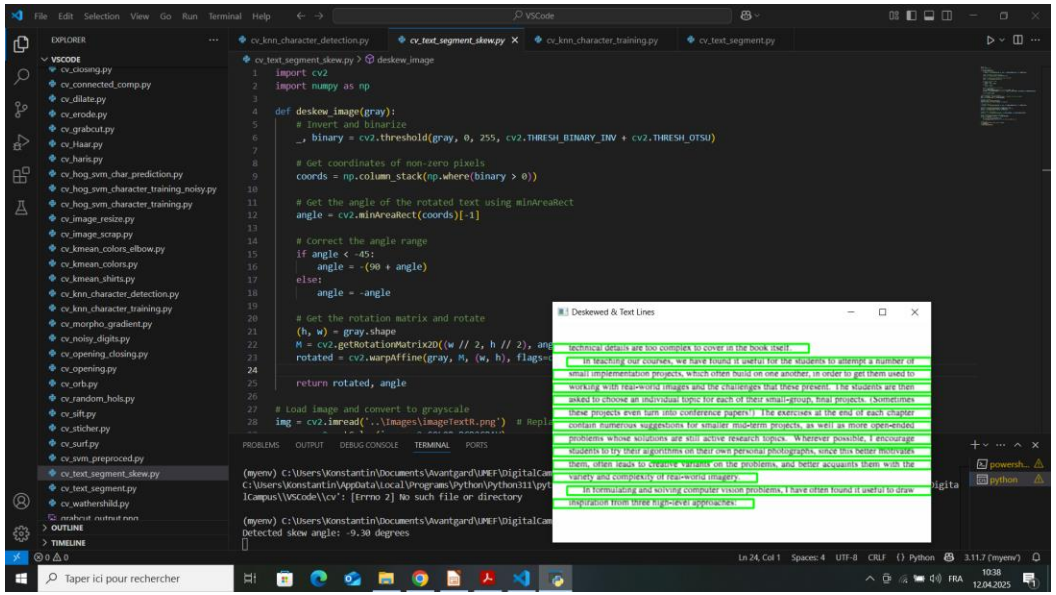
In formulating and solving computer vision problems, I have often found it useful to draw inspiration from three high-level approaches:

Exercise from previous session 2/3

```
    # Correct the angle range
    if angle < -45:
        angle = -(90 + angle)
    else:
        angle = -angle
    # Get the rotation matrix and rotate
    (h, w) = gray.shape
    M = cv2.getRotationMatrix2D((w // 2, h // 2), angle, 1.0)
    rotated = cv2.warpAffine(gray, M, (w, h), flags=cv2.INTER_CUBIC, borderMode=cv2.BORDER_REPLICATE
    return rotated, angle

# Load image and convert to grayscale
img = cv2.imread('..\Images\imageTextR.png')  # Replace with your own image
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Deskew the image
deskewed, angle = deskew_image(gray)
print(f"Detected skew angle: {angle:.2f} degrees")
```

Exercise from previous session 3/3

```
# Threshold again
_, binary = cv2.threshold(deskewed, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# Dilation with horizontal kernel to connect text lines
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (50, 1))
dilated = cv2.dilate(binary, kernel, iterations=1)

# Find contours (text lines)
contours, _ = cv2.findContours(dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Draw bounding boxes on the rotated image
result = cv2.cvtColor(deskewed, cv2.COLOR_GRAY2BGR)
for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)
    cv2.rectangle(result, (x, y), (x + w, y + h), (0, 255, 0), 2)

# Show result
cv2.imshow("Deskewed & Text Lines", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Introduction to Deep Learning

Deep Learning is a subfield of machine learning that focuses on algorithms inspired by the structure and function of the human brain, known as artificial neural networks. It involves training models with many layers (hence "deep") to automatically learn complex patterns and representations from large amounts of data.

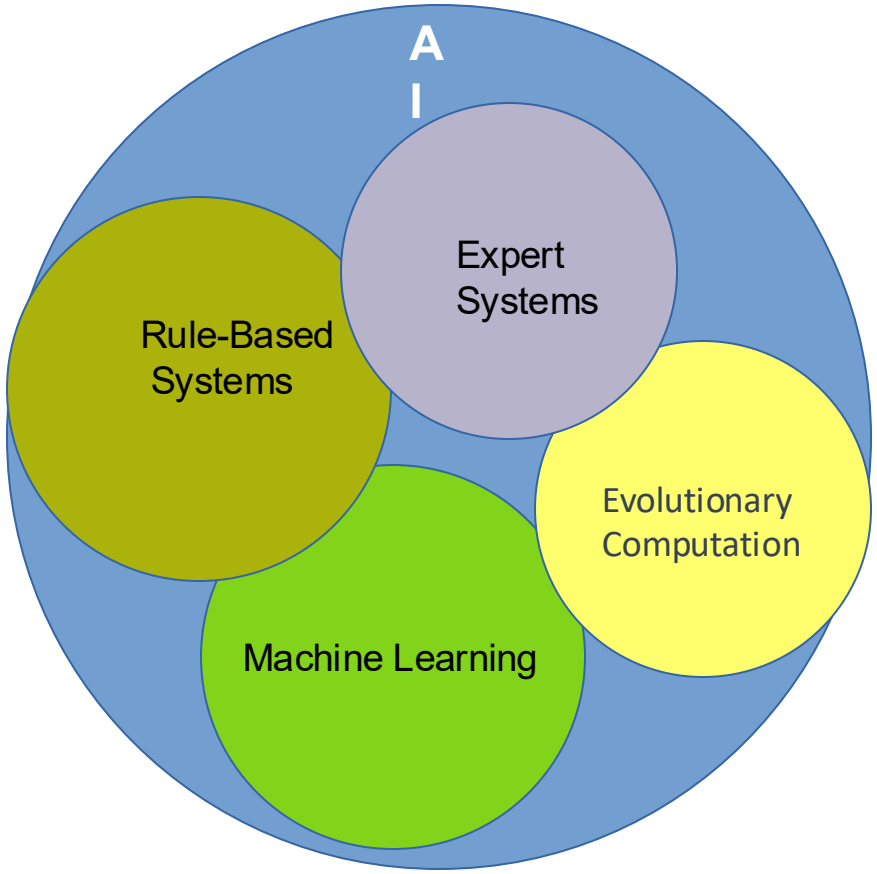
Key Characteristics:
Neural Networks: Deep learning models are built using layers of neurons (nodes) that mimic how the brain processes information.

Multiple Layers: These networks have multiple layers between input and output (e.g., Convolutional Layers, Recurrent Layers, Fully Connected Layers).

Feature Learning: Automatically learns features from raw data (e.g., images, text, audio) — no need for manual feature engineering.

High Performance: Particularly powerful for tasks involving vision, speech, natural language, and autonomous systems.

Machine Learning and AI



Rule-Based Systems: operate on a set of predefined rules. They follow logical statements and if-else conditions to make decisions.

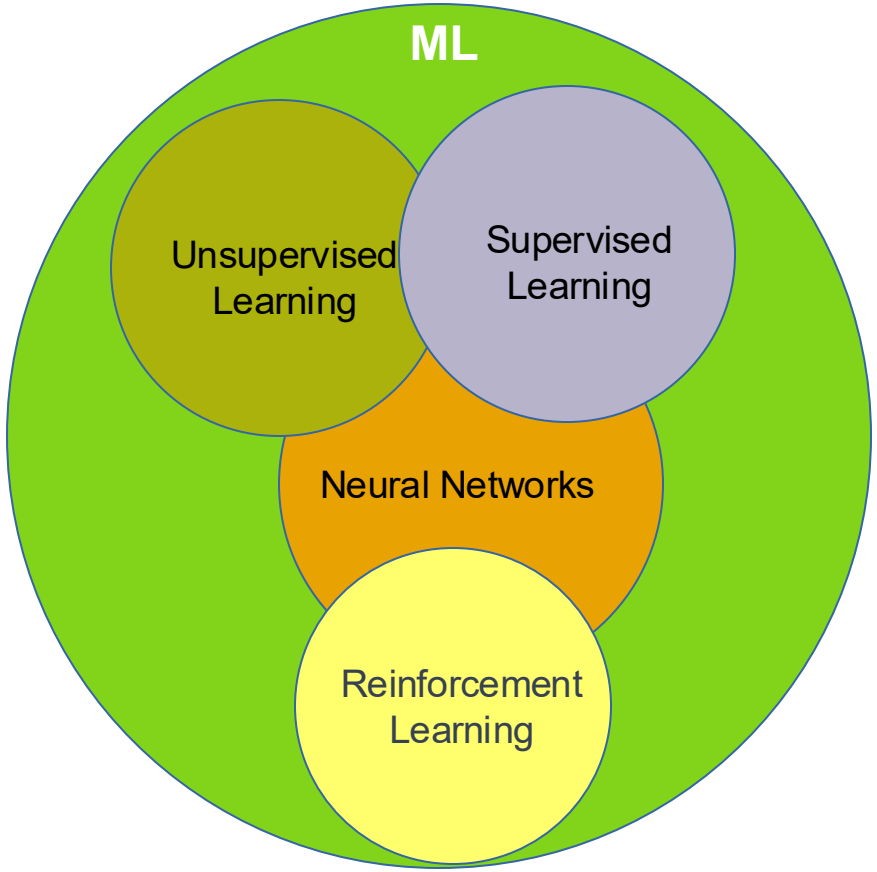
Expert Systems: AI systems designed to emulate and mimic human expertise in a specific domain. They use knowledge bases and inference engines.
vague or imprecise information.

Evolutionary Computation: Algorithms inspired by the principles of biological evolution, involving the selection and adaptation of solutions over generations.

Machine Learning: Systems that learn from data without being explicitly programmed. It involves the development of models that can generalize patterns from input data.

Fuzzy Logic: Systems that deal with uncertainty and approximate reasoning, allowing for the representation of vague or imprecise information.

Machine Learning



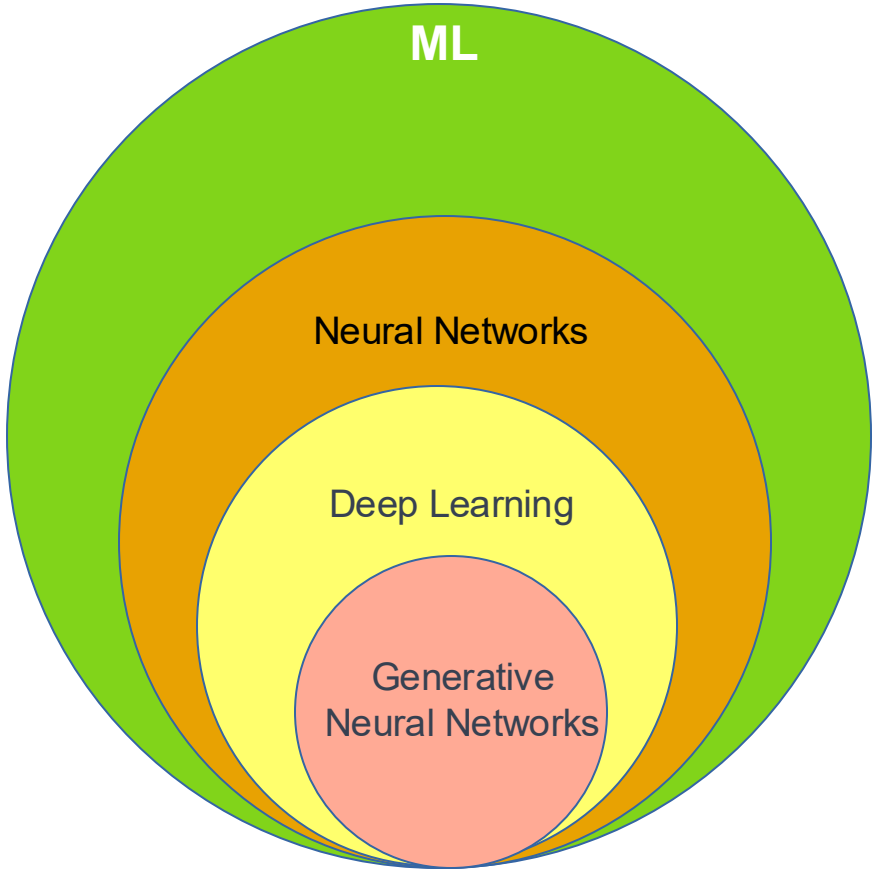
Unsupervised Learning: Learning from unlabeled data, finding patterns and relationships within the data.

Supervised Learning: Learning from labeled data, where the algorithm is trained on input-output pairs.

Reinforcement Learning: Learning through trial and error, where the algorithm receives feedback in the form of rewards or penalties.

Neural Networks: Systems inspired by the structure and function of the human brain, composed of interconnected nodes (neurons) organized in layers.

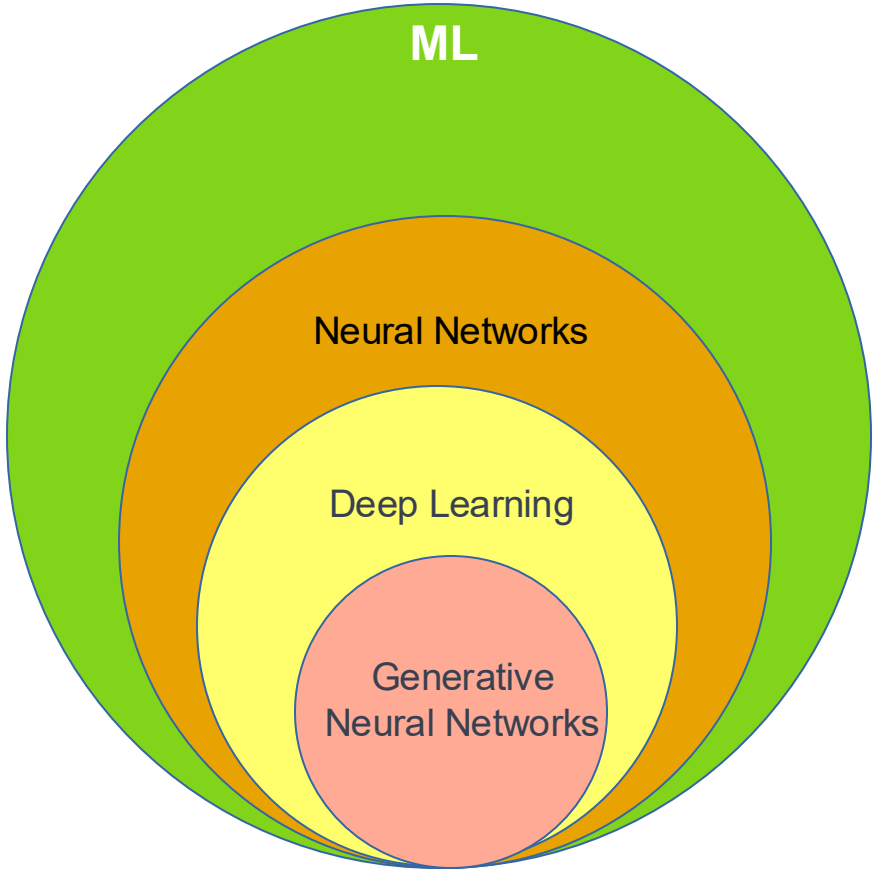
Neural Networks



Deep Learning: involves the development and application of artificial neural networks with multiple layers (deep neural networks) to analyze and extract patterns from large sets of data (Big Data).

Generative Neural Networks: focus on generating new data instances that resemble a given training dataset.

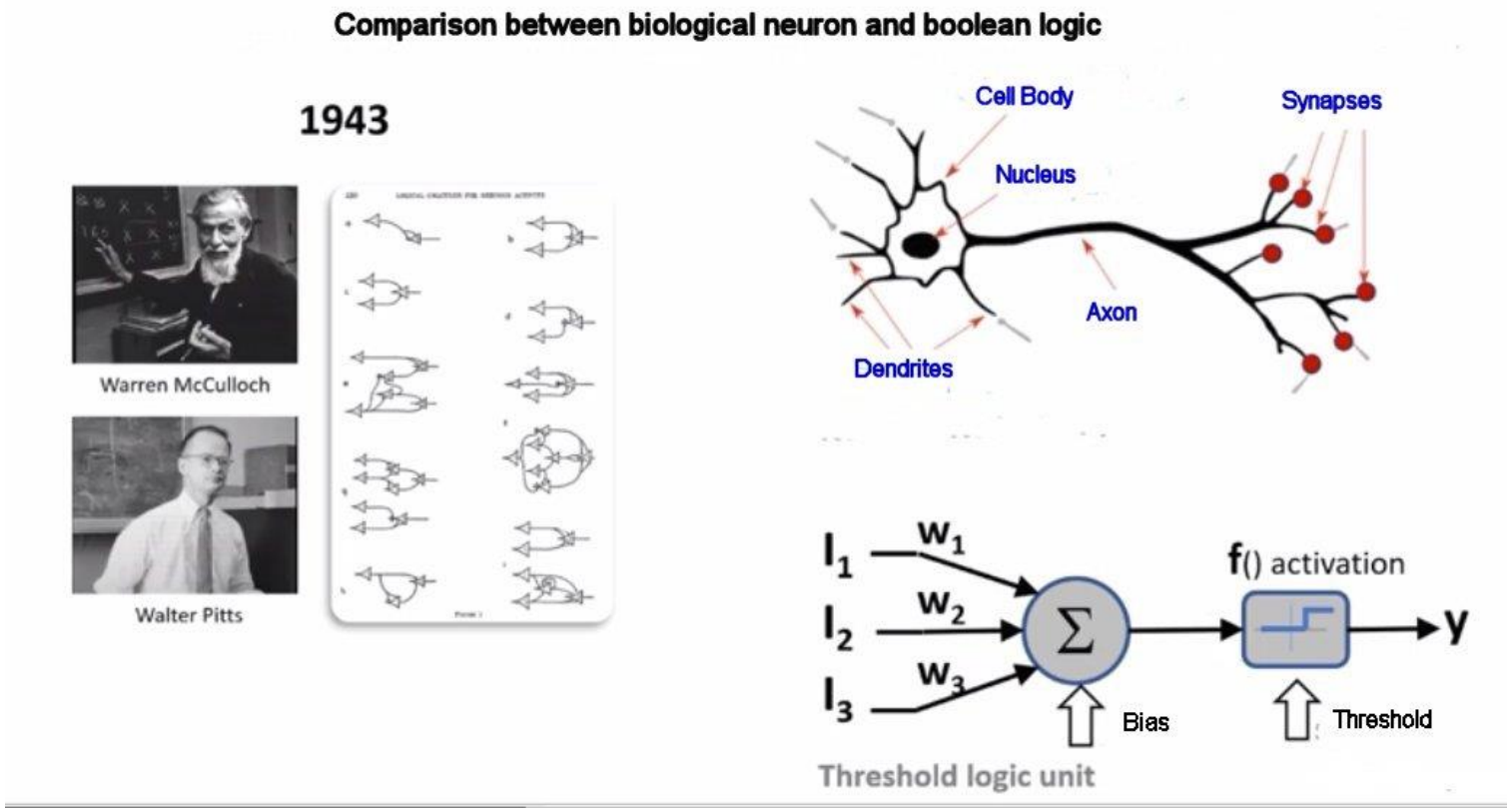
Neural Networks



Deep Learning: involves the development and application of artificial neural networks with multiple layers (deep neural networks) to analyze and extract patterns from large sets of data (Big Data).

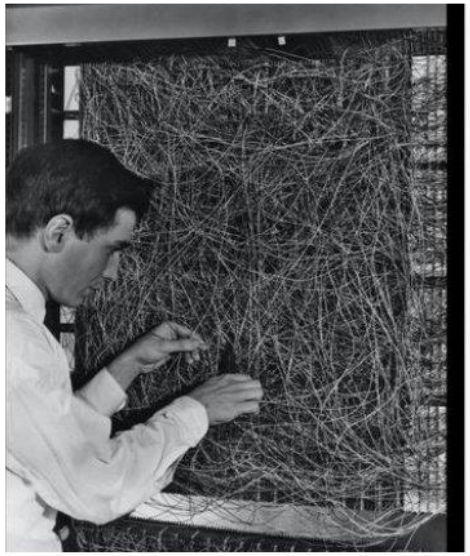
Generative Neural Networks: focus on generating new data instances that resemble a given training dataset.

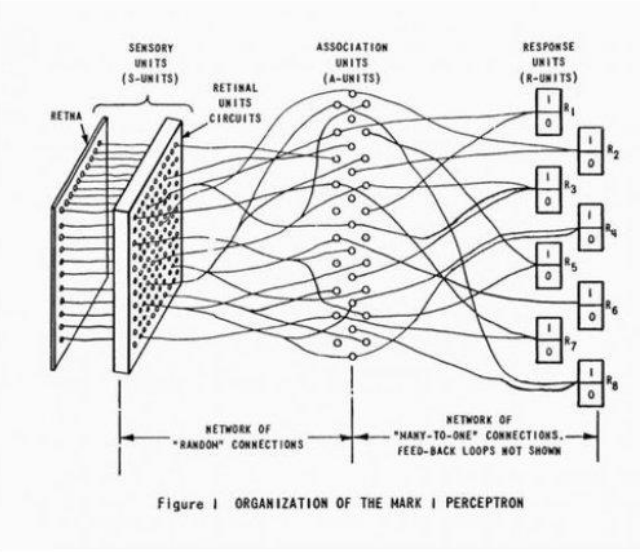
Artificial Neuron



Frank Rosenblatt - Perceptron

In 1957, Frank Rosenblatt developed the “perceptron”, a model composed of a layer of input neurons and another as output, initially simulated on an IBM 704 computer at the Cornell Aeronautical Laboratory. Designed to recognize shapes, it constitutes the first system capable of learning from experience. And the first real artificial neural network.

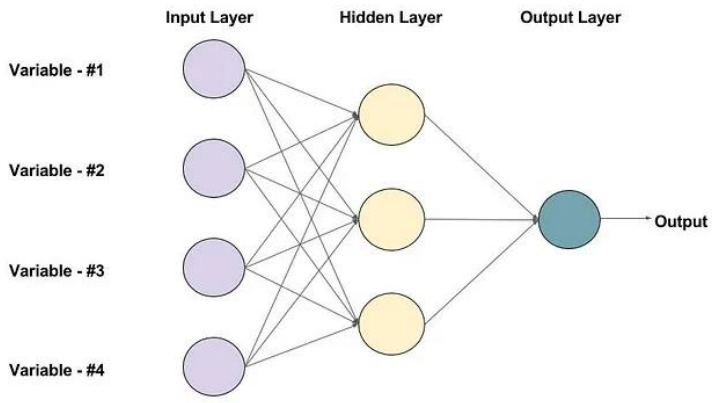




Frank Rosenblatt, Getty Images

What is a Layer in Artificial Neural Networks ?

In neural networks, a layer is a collection of neurons (units) that process inputs and pass the result to the next layer.
Input layer receives the raw data.
Hidden layers do most of the learning.
Output layer gives the final prediction.



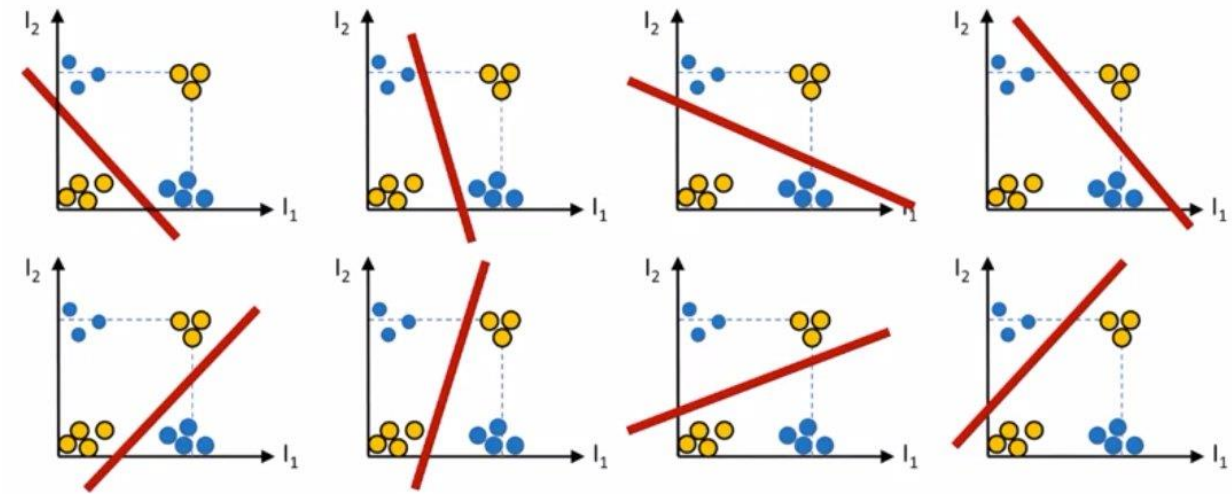
An example of a Feed-forward Neural Network with one hidden layer (with 3 neurons)

The Perceptron is a linear classifier

In 1962, Marvin Minsky and Seymour Papert highlighted the problem of linear transformation.
Perceptrons, An Introduction to Computational Geometry, by Marvin Minsky and Seymour Papert. MIT Press, Cambridge, Mass., 1969.

First winter in AI: 1974 – 1980:

Marvin Minsky, Wikipedia

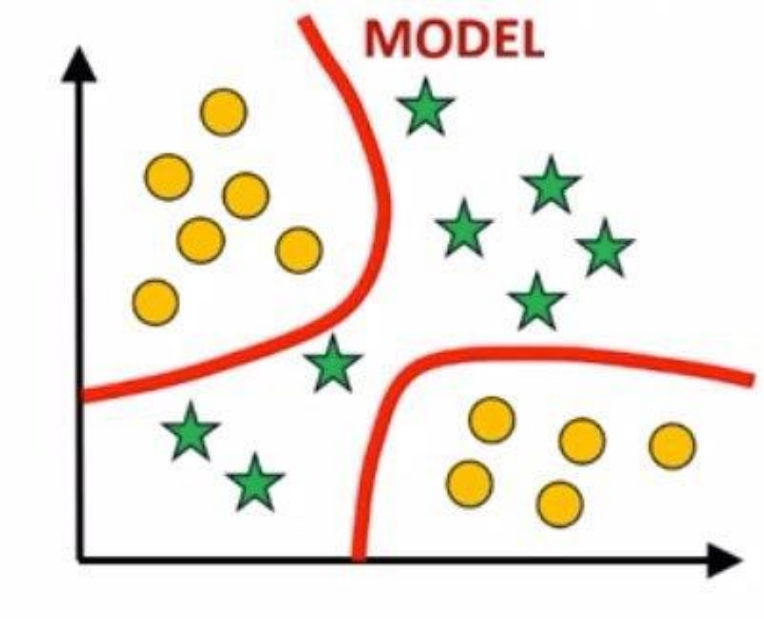


Source: jsilab.ch

Nonlinear Neural Networks

A nonlinear neural network is a neural network that uses nonlinear transformations in its layers, such as nonlinear activation functions, convolution, or pooling.

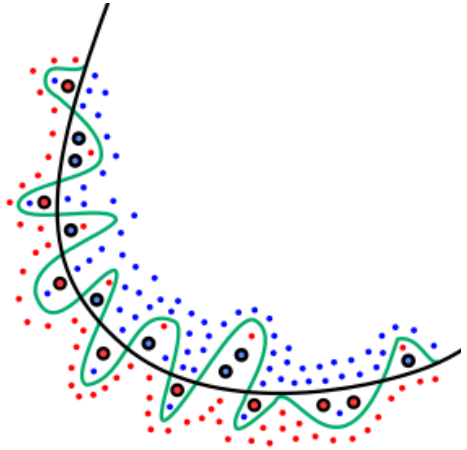
Problem - the network becomes unstable with the increase in the number of neurons (max 12 neurons).
The second winter of AI: 1987 – 1993.

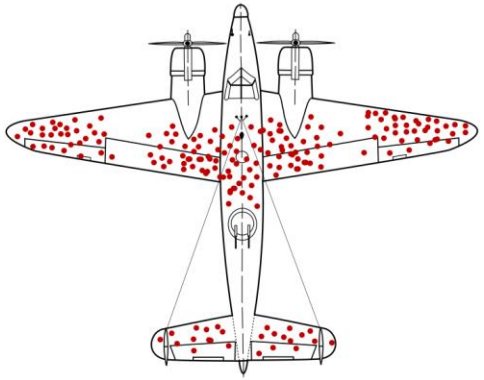


Source: jsilab.ch

Problems of Neural Networks

1. **Convergence**: The learning process continues until the model reaches convergence, where further tuning so variables of model do not significantly reduce the error on the training data.
2. **Overfitting and Underfitting**: Proper tuning of model parameters is crucial to avoid overfitting or underfitting. Overfitting occurs when a model learns the training data too well but struggles with new, unseen data. This is often a result of overly complex models. Underfitting occurs when a model is too simple to capture the underlying patterns in the data
3. **Robustness**: Neural networks may lack robustness to variations in input data, particularly in case of overfitting.
4. **Data Dependency**: Neural networks heavily rely on large amounts of labeled training data. Insufficient, biased or low quality data can lead to poor generalization. Labeling could be time consuming and expensive.
5. **Interpretability**: Neural networks are like black-box models, making it challenging to interpret and understand how they make specific decisions. They lack a deep understanding of causality and may learn correlations in data without grasping the underlying causal relationships.

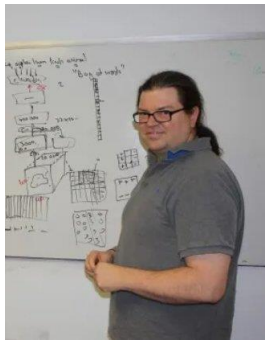




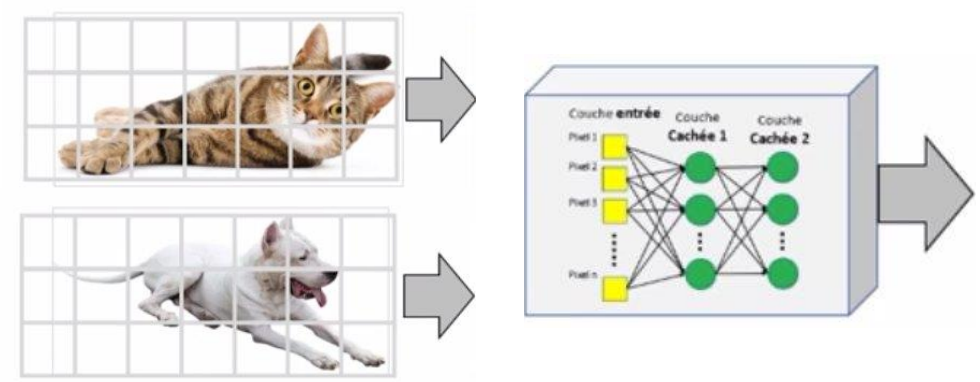
Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) - feedforward neural network with the application of convolution kernels (or cross-correlation) in cascade. CNNs are used for tasks involving grid-like data, such as images and videos.

Structured in layers similar to the human brain!



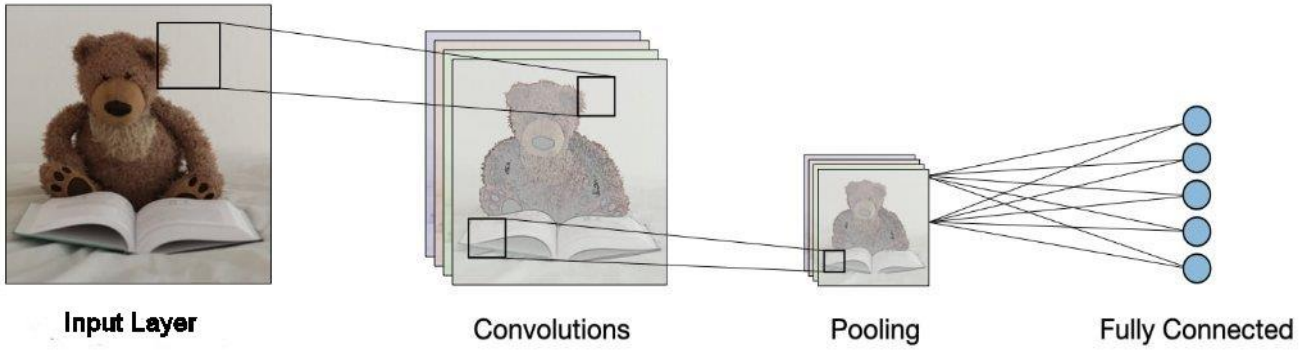
Yann LeCun 1990



Cat (0.85)
Dog (0.91)
……..

Types of Layers in CNNs

Convolutional Layer: apply convolution operations (correlation) using learnable filters (kernels) to detect patterns and features in the input data. Multiple filters are used to capture different features, forming feature maps.
Pooling layers: perform down-sampling or subsampling by reducing the spatial dimensions of the feature maps. Max pooling and average pooling are common operations in which the maximum or average value is taken from a group of neighboring pixels.
Fully Connected Layer: connect every neuron from one layer to every neuron in the next layer.



https://stanford.edu/

Fundamental Difference with the Old Neural Networks

Neurons are structured in multiple layers with specific functions similar to the human brain!

Eureka! K. Startchev and NightCafe

Model

Definition:
A model is a function (or set of functions) that maps input data to output predictions using parameters learned from training data.

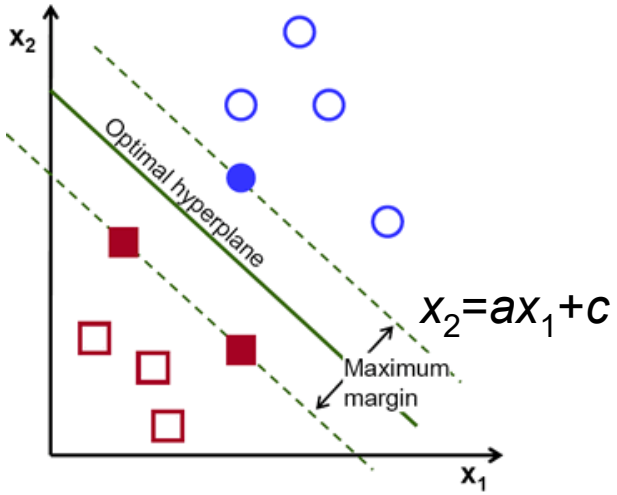
Think of it like this:

"Model" = Mathematical Machine
Input: Features (e.g., pixel values, numbers, text)
Process: Applies weights, biases, and activations (these are parameters)
Output: A prediction (e.g., digit label, cat vs. dog, sentiment, etc.)

Features and Parameters in CNN model

In the past sessions we have defined features as small patches of images or image properties that contains characteristic properties of objects in the scene. The main goal of Computer Vision Scientist was to find and propose such reliable features like Haar or HOG. In CNN however we left the algorithm to find the features. We start with sending all pixels to the first layer then each layer transforms input features into new, learned features.

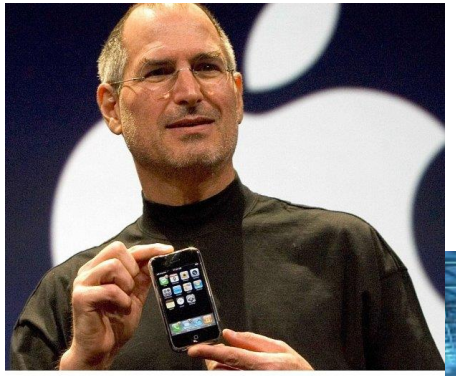
Model parameters are the values (weights and biases) the model learns during training. These weights are applied to the inputs to compute outputs layer by layer. Therefore parameters transform features into meaningful representations



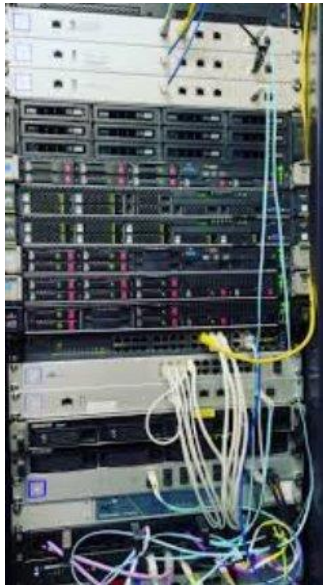
https://docs.opencv.org/4.x/d4/db1/tutorial_py_svm_basics.html

Deep Learning

We have described a neural network capable of using a large number of neurons, but we also need a large number of data and enormous computing power to train this network.



Netscape (1994)
Smatphone (2007)
Storage space 2003
Servers (Amazon) (2006)



graphic processor optimised for deep learning

processor of 320000 neurons for computer vision application

Deep Learning fields of Application

Classification
Character recognition (OCR)
Healthcare.
Robotics, autonomous cars.
Image classification.
Visual recognition.
Natural language processing (NLP), translation.
Fraud detection.
Demographic and electoral forecasts.
Algorithmic Trading and Investment Strategies.
Robotic Process Automation (RPA)

Generation
Creation of images, music, programming, fiction, films.
Optical replication, deepfake, digital twins.

TensorFlow

TensorFlow is an open-source platform developed by Google Brain team for building and deploying machine learning and deep learning models. It supports everything from simple models for beginners to massive production-grade systems.

TensorFlow can be used in a wide variety of programming languages, including Python, JavaScript, C++, and Java, facilitating its use in a range of applications in many sectors.

TensorFlow can run on multiple CPUs and GPUs (with optional CUDA and SYCL extensions for general-purpose computing on graphics processing units). TensorFlow is available on 64-bit Linux, macOS, Windows, and mobile computing platforms including Android and iOS.

TensorFlow computations are expressed as stateful dataflow graphs. The name TensorFlow derives from the operations that such neural networks perform on multidimensional data arrays, which are referred to as tensors. During the Google I/O Conference in June 2016, Jeff Dean stated that 1,500 repositories on GitHub mentioned TensorFlow, of which only 5 were from Google. (Wikipedia)

TensorFlow – Core Concepts

1. **Tensors**: Multi-dimensional arrays (like NumPy arrays) are the basic data structure used in TensorFlow.
2. **Graphs**: Computation is represented as a directed graph of operations (low-level TensorFlow).
3. **Keras** API: High-level, user-friendly API built into TensorFlow for fast prototyping and training.
4. **Operations**: Nodes in the graph, performing math (e.g., matrix multiplication, convolution, etc.).
5. **End-to-End M**L: From data preprocessing to training and deployment.
6. **Scalability**: Easily runs on CPUs, GPUs, TPUs, mobile, and cloud.

TensorFlow – Use Cases

Field	Application
🖼️ Computer Vision	Image classification, object detection, segmentation
🗣️ Natural Language Processing	Translation, chatbots, sentiment analysis
🧬 Healthcare	Disease prediction, drug discovery
📊 Finance	Fraud detection, time series prediction
🤖 Robotics	Sensor fusion, autonomous navigation
🧪 Research	Prototyping complex ML experiments

TensorFlow – Basic Workflow

1. **Define the model**: defining the entire architecture of the model

2. **Compile the model**: tells the model how to learn (how to adjust its internal weights during training and how to evaluate its performance)

3. **Train the model**: optimise parameters.

4. **Predict**.

TensorFlow – Models

In TensorFlow, especially using its high-level Keras API, a model is the heart of your machine learning system. A TensorFlow model is essentially a container for:
Layers – the building blocks (dense, conv, pooling, etc.)
Weights – the trainable parameters inside layers
Forward pass logic – how input data moves through layers
Training configuration – optimizer, loss function, metrics

Define a model - defining the entire architecture of the model, including:

Input layer	Shape of incoming data
Hidden layers	Number of layers and their types (Dense, Conv2D, etc.)
Activation functions	For each layer (e.g., relu, softmax, sigmoid)
Output layer	Number of outputs and activation for final prediction
Layer order and flow	The exact computation pipeline

TensorFlow – example of model

```
model = tf.keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

You are defining all layers dense - A fully connected layer (every input connects to every output unit):
Input layer — expects a vector of size 784
First hidden layer — The layer has 128 neurons (units) and uses the ReLU activation function, which adds non-linearity
Second hidden layer — 64 units + ReLU
Output layer — 10 units (classes) + Softmax activation

TensorFlow – Compile the Model

Before training, the model needs to be compiled with:

optimizer → how the model updates weights (e.g., Adam, SGD). An optimizer controls how the model updates its weights (parameters) based on the loss function. It's the engine that tells the model "how to get better" by minimizing errors. How It Works (in simple terms):

The model makes a prediction
Loss is calculated (how wrong it was)
The optimizer adjusts weights to reduce that loss using gradients

loss → Tells the model how wrong its predictions are — it's what the optimizer minimizes during training.

metrics → Used to evaluate and monitor model performance — helpful for humans, but not directly optimized by the model.

TensorFlow – Train the Model

Train the Model:
model.fit(x_train, y_train, epochs=10, batch_size=32, validation_split=0.1)
x_train - data,
y_train - labels,
An epoch is one full pass through the entire training dataset. More epochs = more optimisation, but risk of overfitting.
batch_size: Data is not fed to the model all at once, but in mini-batches. batch_size=32 means the model sees 32 samples at a time, updates weights, and then moves to the next 32. It balances speed and learning quality.
validation_split=0.1. This reserves 10% of the training data for validation — to check how well the model is doing on unseen data during training. It's useful for: Detecting overfitting and tuning hyperparameters.

Instalation of TensorFlow

Before installing TensorFlow make sure your system has the Microsoft Visual C++ Redistributable. If they are missing downlod from:

https://learn.microsoft.com/en-us/cpp/windows/latest-supported-vc-redist?view=msvc-170#latest-microsoft-visual-c-redistributable-version

Run the instaler and when finished restart your computer.

Then instal TensorFlow as usual with:

pip install tensorflow

NB! TensorFlow works with Python version lower then 3.12. If you're using Python 3.12 or higher, that could be the problem. Downgrade to Python 3.10 or 3.11

CNN with TensorFlow code example - 1/3

Convolutional Neural Network (CNN) for MNIST digit classification in TensorFlow/Keras:
MNIST: Dataset of 70,000 grayscale images of handwritten digits (28x28 pixels).

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# 1. Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# 2. Preprocess the data
x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

CNN with TensorFlow code example 2/3

```
# 3. Define the CNN model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),  # Conv Layer 1 Learns local patterns

                                    like edges and shapes. 32 kernels 3x3
    layers.MaxPooling2D((2, 2)),                    # Pooling Reduces computation, most important features.
                                                                                                    # Reduces
spatial size by taking the max in each 2×2 patch.
    layers.Conv2D(64, (3, 3), activation='relu'),        # Conv Layer 2 with 64 filters of size 3×3.
                                                                                                    # Extracts
more abstract patterns.
    layers.MaxPooling2D((2, 2)),                     # Pooling Again halves spatial dimensions
    layers.Flatten(),                               # Flatten to 1D - Final classifier layers.
    layers.Dense(128, activation='relu'),            # FC layer A fully connected layer with 128 neurons.
    layers.Dense(10, activation='softmax')           # Output layer  Gives class probabilities for digits 0–9.
                                                                                                    # Softmax: Converts
raw scores into probabilities that sum to 1.
])

# 4. Compile the model
model.compile(optimizer='adam',            #most popular and effective optimizers in deep learning
```

CNN with TensorFlow code example 3/3

```
# 5. Train the model
model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.1)

model.save('..\Models\TensorFlow\my_model.h5')

# 6. Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"\nTest accuracy: {test_acc:.4f}")
```

>> Test accuracy: 0.9910

CNN with TensorFlow, predict digit code example 1/2

```
import cv2
import numpy as np
from tensorflow.keras.models import load_model

# Load the trained model
model = load_model("..\Models\TensorFlow\my_model.h5")

# Load your image in grayscale
img = cv2.imread("..\Images\dign_1.png", cv2.IMREAD_GRAYSCALE)

# Resize to 28x28 (MNIST format)
img = cv2.resize(img, (28, 28))

# Invert colors if background is black and digit is white (MNIST is white digit on black)
#img = cv2.bitwise_not(img)

# Normalize the pixel values
img = img.astype("float32") / 255.0
```

CNN with TensorFlow, predict digit code example 2/2

```
# Reshape to match model input: expects 4D input: (batch_size, height, width, channels)
img = np.expand_dims(img, axis=0)  # add batch dimension
img = np.expand_dims(img, axis=-1)  # add channel dimension

# Predict
predictions = model.predict(img)
predicted_class = np.argmax(predictions)

print(f"Predicted digit: {predicted_class}")
```

```
>>Predicted digit: 1
>>Class probabilities: [1.1225038e-05 9.6245849e-01 1.1890194e-04 1.0716126e-05 7.4624244e-05
 2.8131282e-07 7.0207861e-06 3.7217092e-02 3.9089180e-05 6.2603642e-05]
```

Person detection with Use SSD Mobilenet V2 from TensorFlow Hub

What is TensorFlow Hub?
TensorFlow Hub (TF Hub) is a repository of reusable machine learning models. Models trained on large datasets like ImageNet or COCO.Think of it as a giant library of pretrained models for:

1. Image classification
2. Object detection
3. Text embedding / NLP
4. Audio recognition
5. Style transfer
6. Segmentation

You can use these models without training anything from scratch — just plug and play.

TensorFlow Hub models

Browse available models here:
https://tfhub.dev

You'll find models like:
https://tfhub.dev/google/imagenet/mobilenet_v2_100_224/classification/5
https://tfhub.dev/google/ssd_mobilenet_v2/2 (object detection)
https://tfhub.dev/google/universal-sentence-encoder/4 (NLP)

TF Hub models are provided as SavedModels (a TensorFlow format), so they're easy to:
Fine-tune on your own data
Use in production
Export to TensorFlow Lite or TF.js

SSD MobileNet V2

SSD MobileNet V2 is a real-time object detection model that combines:

1. MobileNetV2 = A lightweight, efficient CNN used as the backbone, extracts features from the input image
2. SSD = Single Shot Multibox Detector (a fast object detection algorithm) predicts:
   - Which objects are present
   - Where they are (bounding boxes)
   - How confident it is about each prediction

Together, they form a fast and efficient model for detecting objects in images or videos — great for mobile devices or real-time applications. It has around 90 classes in total like: person, dog, cat, car, bicycle, traffic light, etc.

Person detection with Use SSD Mobilenet V2 code example 1/3

```
import tensorflow as tf
import tensorflow_hub as hub
import numpy as np
import cv2

# Load SSD model from TF Hub
detector = hub.load("https://tfhub.dev/tensorflow/ssd_mobilenet_v2/2")

# Load labels (COCO labels - person = 1)
LABELS = {1: 'person'}  # COCO label 1 is 'person' (you could also load full list)

# Load and prepare image
img = cv2.imread("..\Images\lena.jpg")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# The model expects uint8 and a batch dimension
img_resized = tf.image.resize_with_pad(img_rgb, 320, 320)
img_resized = tf.cast(img_resized, tf.uint8)
input_tensor = tf.expand_dims(img_resized, 0)
```

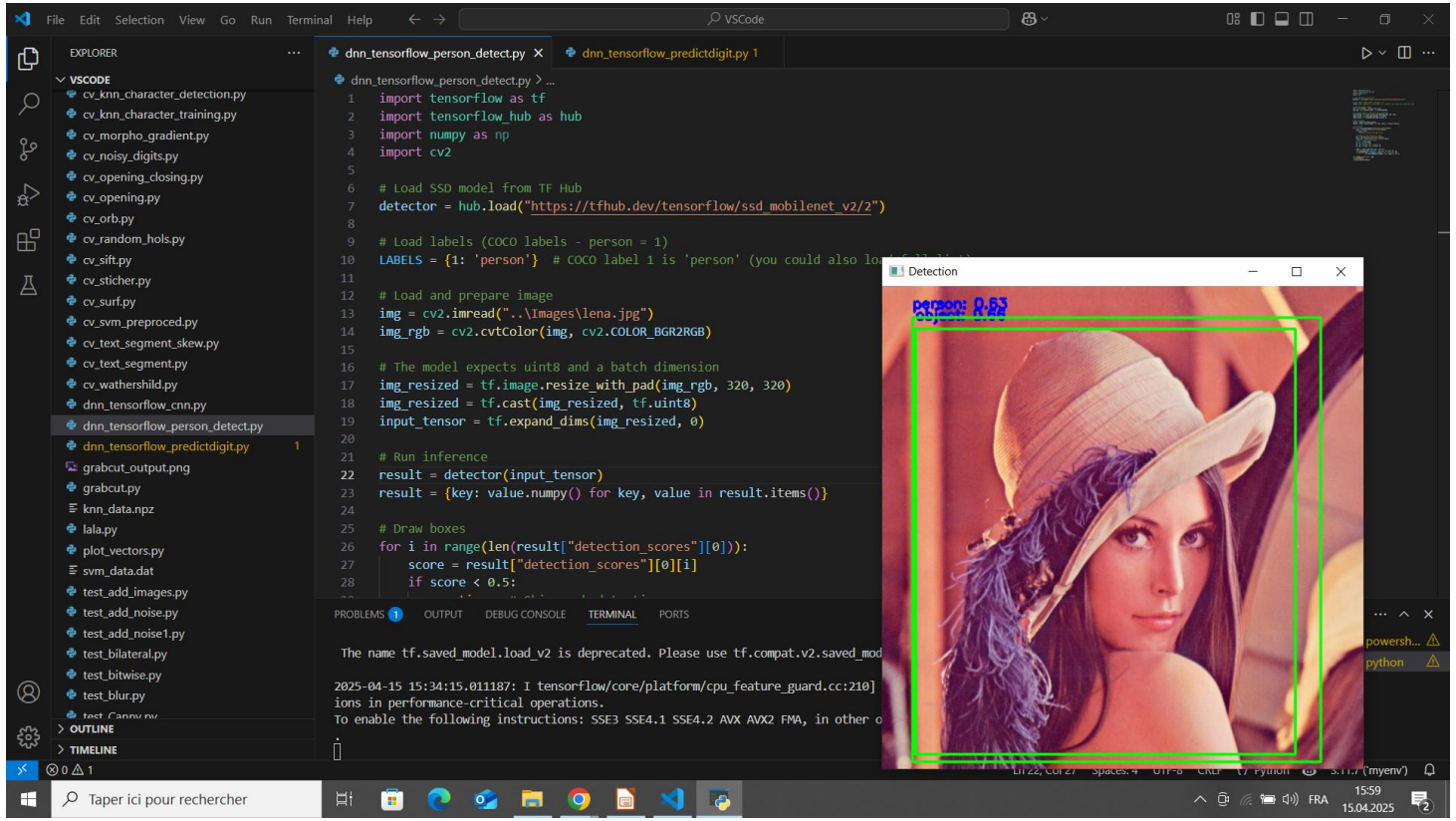
Person detection with Use SSD Mobilenet V2 code example 2/3

```
# Run inference
result = detector(input_tensor)
result = {key: value.numpy() for key, value in result.items()}

# Draw boxes
for i in range(len(result["detection_scores"][0])):
    score = result["detection_scores"][0][i]
    if score < 0.5:
        continue  # Skip weak detections

    # Proceed to draw the box or label
    box = result["detection_boxes"][0][i]
    class_id = result["detection_classes"][0][i]

# Draw or print result here
    h, w, _ = img.shape
    y1, x1, y2, x2 = box
    x1, x2 = int(x1 * w), int(x2 * w)
    y1, y2 = int(y1 * h), int(y2 * h)
```

Person detection with Use SSD Mobilenet V2 code example 3/3

```
label = LABELS.get(class_id, "object")
cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
cv2.putText(img, f"{label}: {score:.2f}", (x1, y1 - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)
```

```
cv2.imshow("Detection", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Real-Time Person detection with Use SSD Mobilenet V2 code example 1/2

```
import tensorflow as tf
import tensorflow_hub as hub
import numpy as np
import cv2

# Load SSD MobileNet V2 from TF Hub
print("Loading model...")
detector = hub.load("https://tfhub.dev/tensorflow/ssd_mobilenet_v2/2")
print("Model loaded.")
# Open webcam
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
   # Convert BGR to RGB
    img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    img_resized = tf.image.resize_with_pad(img_rgb, 320, 320)
    img_resized = tf.cast(img_resized, tf.uint8)
```

Real-Time Person detection with Use SSD Mobilenet V2 code example 1/2

```
input_tensor = tf.expand_dims(img_resized, 0)

# Run detection
result = detector(input_tensor)
result = {key: value.numpy() for key, value in result.items()}

# Extract data
boxes = result['detection_boxes'][0]
classes = result['detection_classes'][0].astype(np.int32)
scores = result['detection_scores'][0]

# Draw detections
height, width, _ = frame.shape
for i in range(len(scores)):
    if scores[i] < 0.5:
        continue
```

Real-Time Person detection with Use SSD Mobilenet V2 code example 2/2

```
        # Get box coordinates (normalized → absolute)
        y1, x1, y2, x2 = boxes[i]
        x1 = int(x1 * width)
        x2 = int(x2 * width)
        y1 = int(y1 * height)
        y2 = int(y2 * height)
# Draw box and label
        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
        label = f"ID: {classes[i]}, Conf: {scores[i]:.2f}"
        cv2.putText(frame, label, (x1, y1 - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

    # Show frame
    cv2.imshow("Webcam Object Detection", frame)
    # Exit on 'q' key
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

Exercise

The last proposed code Real-Time Person detection with Use SSD Mobilenet V2 was rather slow. Propose a way (and the code) to improve the speed having in mind that we need only a person detection.

In this session we have covered:

Introduction to Deep Learning & CNNs for Image Processing
Overview of Convolutional Neural Networks (CNNs).
Using TensorFlow/Keras with OpenCV.
Exercise: Building a simple CNN for image classification

In the next session we will covered:

Final Project – Building a Computer Vision Application
Choosing a project (e.g., License Plate Recognition, Object Tracking, Hand Gesture Recognition).
Data collection, processing, and augmentation.
Training a model using OpenCV and Deep Learning.

THANKS !!!